MODULE #3 INTRODUCTION TO OOPS PROGRAMMING

Introduction to C++

1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

Ans:-

No.	Procedural Programming (PP)	Object-Oriented Programming (OOP)
1	Focuses on **functions/procedures**	Focuses on **objects and classes**
2	Uses **top-down approach**	Uses **bottom-up approach**
3	Data and functions are **separate**	Data and functions are **combined**
4	Less **data security**	High security using **encapsulation**
5	No concept of **inheritance**	Supports **inheritance**
6	Limited **code reusability**	High **code reusability**
7	Uses **global data**	Uses **data hiding (private/public)**
8	Hard to manage large programs	Easier to manage large programs
9	Examples: **C, Pascal**	Examples: **C++, Java, Python**
10	Suitable for **small programs**	Suitable for **large & complex programs**

2. List and explain the main advantages of OOP over POP.

Ans:- Here are the **main advantages of OOP over POP**

1. Data Security

OOP uses **encapsulation**, so data is hidden and safe.

---

2. Code Reusability

Using **inheritance**, code can be reused, saving time.

---

3. Easy Maintenance

Programs are easier to update and fix.

---

4. Real-world Representation

OOP models real-life objects (like Student, Car).

---

5. Flexibility

Supports **polymorphism**, so one function can work in many ways.

3. Explain the steps involved in setting up a C++ development environment.

Ans:- **Steps to Set Up C++ Environment**

1. Install a compiler (like **Turbo C++ / MinGW / GCC**)
2. Install an IDE (like **Code::Blocks / VS Code**)
3. Write a simple C++ program
4. Compile the program
5. Run the program

4. What are the main input/output operations in C++? Provide examples.

Ans:- Here are the **main input/output operations in C++**

1. Output Operation (cout)

Used to display output on screen.

Example:

```
#include<iostream>
using namespace std;

int main() {
    cout << "Hello World";
    return 0; }
```

2. Input Operation (cin)

Used to take input from user.

Example:

```
#include<iostream>
using namespace std;

int main() {
int a;     cin
>> a;     cout
<< a;
   return 0; }
```

5.What are the different data types available in C++? Explain with examples.

Ans:- Here are the **different data types in C++ examples**

---

1. Basic (Primitive) Data Types

Used to store simple values.

→**int** → integer

int a = 10;

→**float** → decimal number

float b = 3.14;

→**double** → large decimal

double c = 12.3456; ❼**char**

→single character

char d = 'A';

→**bool** → true/false bool

e = true;

---

2. Derived Data Types

Made from basic data types.

→**Array**

int arr[3] = {1, 2, 3};

→**Pointer**

int *p;

---

3. User-defined Data Types

Created by user. ❼**struct**

```
struct Student {
   int age; };
```

→class

```
class A {
int x; };
```

→enum

```
enum day {Mon, Tue};
```

6. Explain the difference between implicit and explicit type conversion in C++.

Ans:-

No.	Implicit Type Conversion	Explicit Type Conversion
1	**Done automatically by the compiler**	**Done manually by the programmer**
2	**Also called type promotion**	**Also called type casting**
3	**No special syntax required**	**Requires casting syntax like (type)**
4	**Generally safe**	**May cause data loss**
5	**Happens when mixing different data types**	**Done when programmer wants specific conversion**
6	**Example: int → float automatically**	**Example: (int)10.75 → 10**
7	**Less control over conversion**	**Full control over conversion**
8	**Used in expressions automatically**	**Used intentionally in code**

7. What are the different types of operators in C++? Provide examples of each.

Ans:- **1. Arithmetic Operators**

Used for calculations +
- * / %

Example:

```
int a = 10, b = 5; cout
<< a + b;   // 15
```

---

2. Relational Operators

Used for comparison ==
!= > < >= <=

Example:

```
cout << (a > b);   // 1 (true)
```

---

3. Logical Operators

Used for logical conditions &&
|| !

Example:

```
cout << (a > 5 && b < 10);
```

---

4. Assignment Operators

Used to assign values
= += -= *= /=

Example:

```
int x = 5; x +=
2;   // x = 7
```

---

5. Increment/Decrement Operators

++ --

Example:

```
int x = 5; x++;
// 6
```

6.Bitwise Operators

Operate on bits
& | ^ ~ << >>

Example:

```
int x = 5 & 3;   // 1
```

---

7. Conditional (Ternary) Operator

?:

Example:

```
int max = (a > b) ? a : b;
```

8. Explain the purpose and use of constants and literals in C++.

Ans:- **Constants**

- **Purpose:** Store values that **cannot be changed**
- Used when value should remain fixed
- Declared using const **Example:**

```
const int x = 10;
```

Literals

- **Purpose:** Fixed values written directly in code
- Used to assign values to variables

Example:

```
int a = 10;

float b = 3.14;

 char c = 'A';
```

9.What are conditional statements in C++? Explain the if-else and switch statements.

Ans:- Used to **make decisions** in a program (execute different code based on condition).

---

1. if-else Statement

Used to check a condition and execute code.

Example:

```
int a = 10;
```

```
if(a > 5) {
    cout << "Greater";
} else {
    cout << "Smaller";
}
```

---

2. switch Statement

Used to select one option from many cases.

Example:

```
int day = 2;

switch(day) {     case 1: cout <<
"Mon"; break;     case 2: cout <<
"Tue"; break;
    default: cout << "Invalid";
}
```

10. What is the difference between for, while, and do-while loops in C++?

Ans:- **for loop**

- Used when **number of iterations is known**
- Condition checked **before** loop

Example:

```
for(int i = 0; i < 5; i++) {
cout << i;
}
```

---

while loop

- Used when **iterations are not fixed**
- Condition checked **before** loop Example:

```
int i = 0; while(i
< 5) {     cout
<< i;     i++;
}
```

---

do-while loop

- Runs **at least once**
- Condition checked **after** loop Example:

```
int i = 0; do
{     cout <<
i;     i++;
} while(i < 5);
```

11. How are break and continue statements used in loops? Provide examples.

Ans:- **break Statement**

- Used to **exit the loop immediately** Example:

```
for(int i = 1; i <= 5; i++) {
if(i == 3) break;       cout
<< i;
}
// Output: 1 2
```

---

continue Statement

- Used to **skip current iteration** and continue next Example:

```
for(int i = 1; i <= 5; i++) {
if(i == 3) continue;     cout
<< i;
}
// Output: 1 2 4 5
```

12. Explain nested control structures with an example.

Ans:- **Nested Control Structures :-**

- Means **using one control structure inside another**
- Example: loop inside loop, or if inside loop

Example (nested loop):

```
for(int i = 1; i <= 3; i++) {
for(int j = 1; j <= 3; j++) {
cout << "*";
    }
    cout << endl;
}
```

Output:

***

***

***

13. What is a function in C++? Explain the concept of function declaration, definition, and calling.

Ans:- A function is a **block of code** that performs a specific task and can be reused.

1. Function Declaration

Tells the compiler about the function (name, return type, parameters)

Example:

```
int add(int, int);
```

---

2. Function Definition

Contains the actual code of the function

- Example:

```
int add(int a, int b) {
   return a + b; }
```

---

3. Function Calling

Used to **execute the function**

- Example:

```
int result = add(5, 3);
```

14. What is the scope of variables in C++? Differentiate between local and global scope.

Ans:- Scope means the **area where a variable can be used** in a program.

No.	Local Scope	Global Scope
1	Variable declared **inside a function or block**	Variable declared **outside all functions**
2	Accessible **only within that function/block**	Accessible **throughout the program**
3	Cannot be used outside its scope	Can be used in any function
4	Created when function is called	Created when program starts
5	Destroyed when function ends	Destroyed when program ends
6	More **secure (limited access)**	Less secure (wide access)

15. Explain recursion in C++ with an example.

Ans:- Recursion is when a **function calls itself** to solve a problem.

Example (Factorial):

```
#include<iostream>
using namespace std;

int fact(int n) {
   if(n == 0)
   return 1;
 else
  return n * fact(n - 1);
}

int main() {
cout << fact(5);
return 0; }
```

16. What are function prototypes in C++? Why are they used?

Ans:- A function prototype is a **declaration of a function** before its use.

Why are they used?

- To **inform the compiler** about the function
- Allows calling a function **before its definition**

- Helps in **error checking** (correct arguments & return type)

17. What are arrays in C++? Explain the difference between singledimensional and multi- dimensional arrays.

Ans:- An array is a **collection of similar data elements** stored in one variable.

Example:

```
int arr[3] = {1, 2, 3};
```

Single-Dimensional Array

- Stores data in **one row (linear)**
- Accessed using **one index**

Multi-Dimensional Array

- Stores data in **rows and columns**
- Accessed using **multiple indices**

18. Explain string handling in C++ with examples.

Ans:- String handling means **working with text (strings)** in C++.

◇ **Example:**

```
#include<iostream> #include<string>
using namespace std;

int main() {
   string name = "Sahil";
   cout << name;
}
```

Common Operations

❼Concatenation (join):

```
string a = "Hello", b = "World"; cout
<< a + b;   // HelloWorld ❼Length:
```

cout << name.length();

❼Access character:

cout << name[0]; // S

❼Input:

getline(cin, name);

19. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

Ans:- Arrays are initialized by **assigning values at the time of declaration**.

1D Array Initialization

int arr[3] = {1, 2, 3};

Access: cout << arr[0];

// 1

2D Array Initialization int

arr[2][2] = {{1, 2}, {3, 4}};

Access:

cout << arr[0][1]; // 2

20. Explain string operations and functions in C++.

Ans:- String operations are used to **work with text** like joining, finding length, etc.

Common String Functions

1. Concatenation (Join Strings)

string a = "Hello";

string b = "World";

```
cout << a + b;   //
HelloWorld
```

---

2. Length string name =
"Sahil"; cout <<
name.length(); // 5

---

3. Access Character cout
<< name[0]; // S

---

4. Input String

```
string name;
getline(cin, name);
```

---

5. Compare Strings

```
if(a == b)
cout << "Same";
```

---

6. Substring string s =
"Hello";

```
cout << s.substr(1,3);
```

21. Explain the key concepts of Object-Oriented Programming (OOP).

Ans:- **Key Concepts of OOP**

1. Class

- A blueprint for objects
  Example: Student

---

2. Object

Instance of a class
Example: s1, s2

---

3. Encapsulation

- Wrapping data and functions together
- Provides **data security**

---

4. Inheritance

- One class can use properties of another
- Helps in **code reuse**

---

5. Polymorphism

- One function, **many forms**
- Example: same function works differently

---

6. Abstraction

- Hides unnecessary details
- Shows only important information

22. What are classes and objects in C++? Provide an example.

Ans:- A class is a **blueprint or template** to create objects. It contains **data (variables)** and **functions**.

Object

An object is an **instance of a class**. It represents a real-world entity.

Example:

```
#include<iostream>
using namespace std;

class Student {
```

```
 public:
    string name;
};

int main() {
    Student s1;
s1.name = "Sahil";
cout << s1.name;
return 0;
```

23. What is inheritance in C++? Explain with an example.

Ans:- Inheritance is a feature where **one class (child) acquires properties of another class (parent)**.

Used for **code reuse**

- **Example:**

```
#include<iostream>
using namespace std;
class Parent { public:
    void show() {
        cout << "This is parent class";
    }
};

class Child : public Parent {
};

int main() {
Child obj;
obj.show();
return 0; }
```

24. What is encapsulation in C++? How is it achieved in classes?

Ans:- Encapsulation means **wrapping data and functions together in a single unit (class)** and **hiding data** from outside.

◇ How is it achieved?

- By using **classes**
- By using **access specifiers**:

  private → data hidden

  public → functions to access data